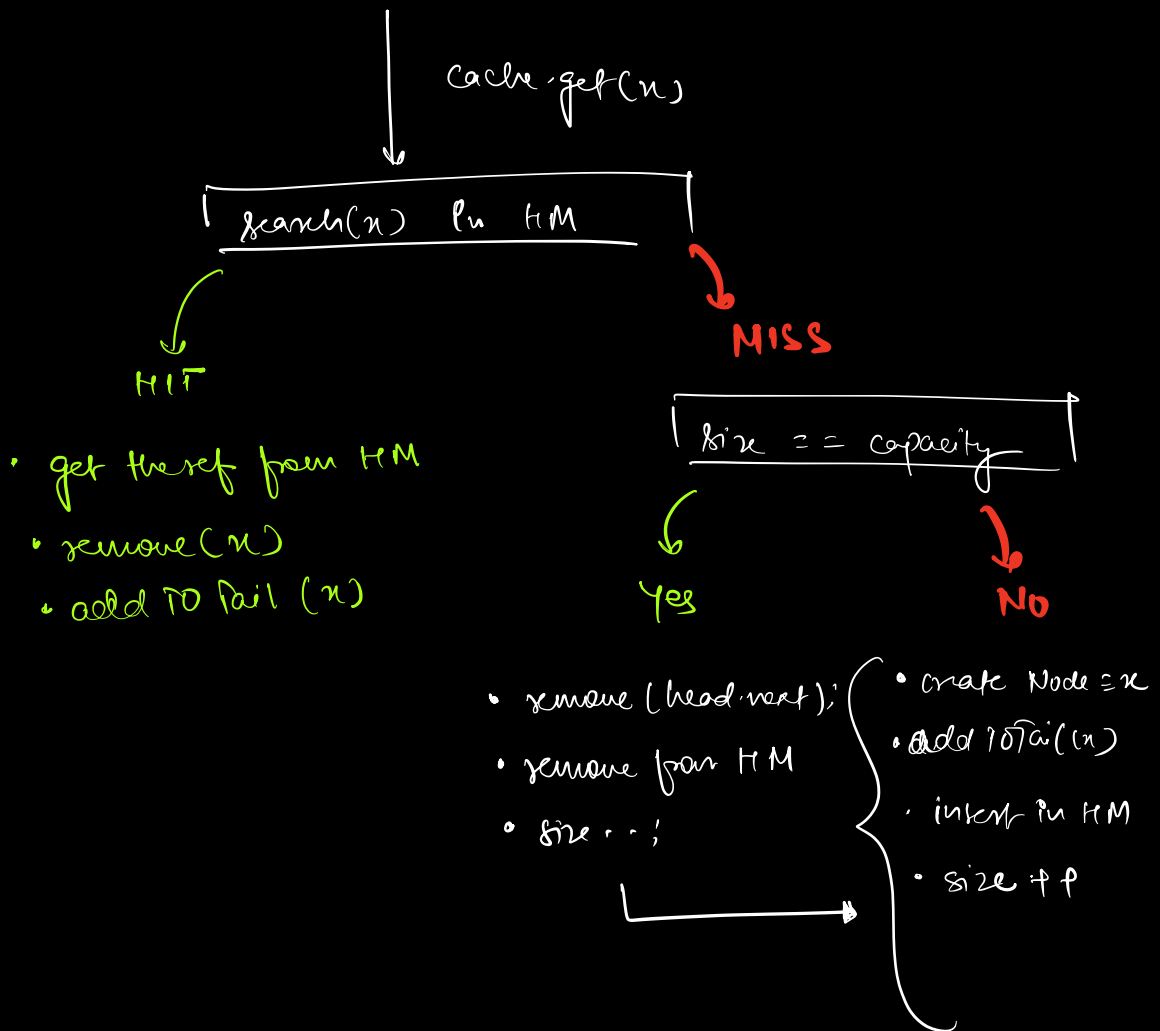




Stack → • data structure

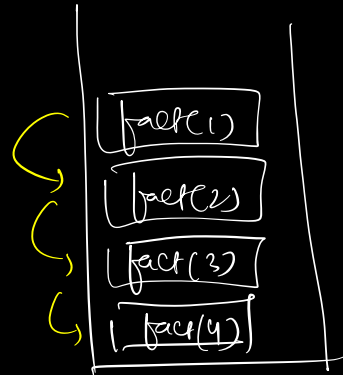
• LIFO → last in first out

Use cases:

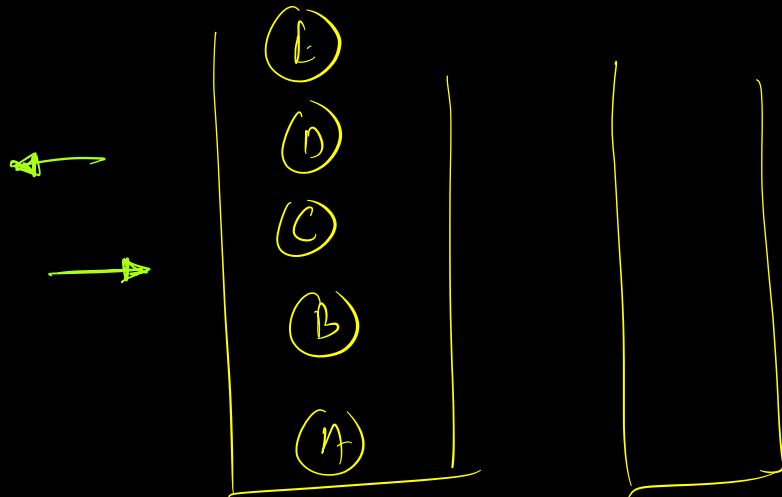
- i) Recursion (Call stack)
- ii) Browser
- iii) Undo (Redo)

→ Recursion

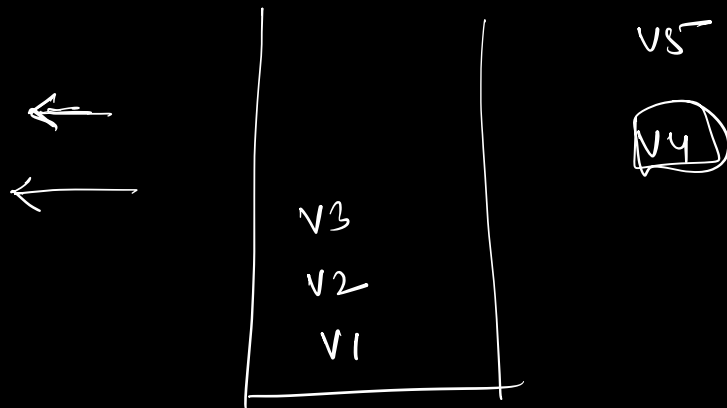
```
int fact(n) {  
    if (n == 0)  
        return 1  
    return n * fact(n-1)  
}
```



ii) browser → navigation (back / forward)



iii) Undo / redo



→ Operations by Stack:-

O(1)

push(x) → Insert x on top of stack

pop() → return and remove top element

top() / peek() → return top element

isEmpty() → return true if empty

size() → size of stack

push 17 ✓

push 5 ✓

push 3 ✓

push 2 ✓

pop → 2

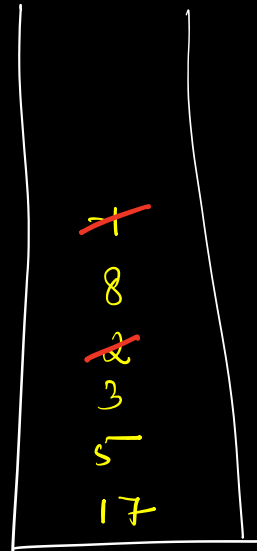
top → 3

push 8

push -1

pop → -1

top → 8



Implementation of Stack :-

array or LL

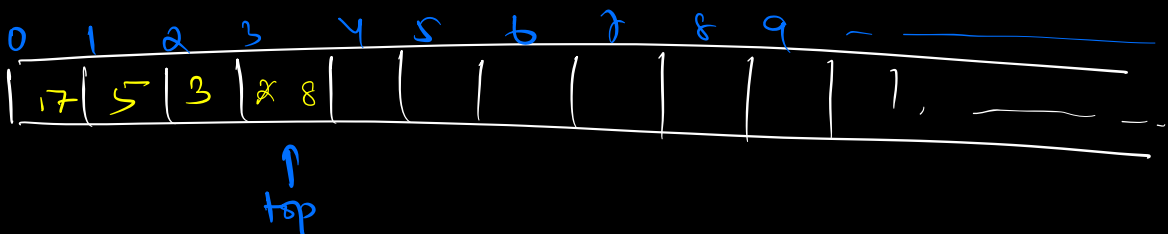
⇒ Array

- user will do max^m → 100 operations

↓

So, size = 100.

size = 0 ; capacity = 100

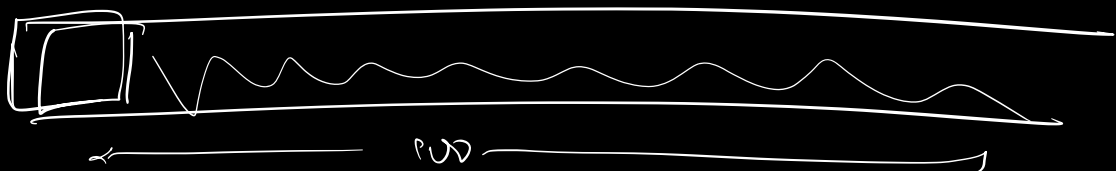


```
push(x) {  
    if (size != capacity) {  
        top++;  
        arr[top] = x;  
        size++;  
    }  
}
```

```
top() {  
    if (top >= 0) {  
        return arr[top];  
    }  
}
```

```
pop() {  
    if (top >= 0) {  
        x = arr[top];  
        top--;  
        size--;  
        return x;  
    }  
}
```

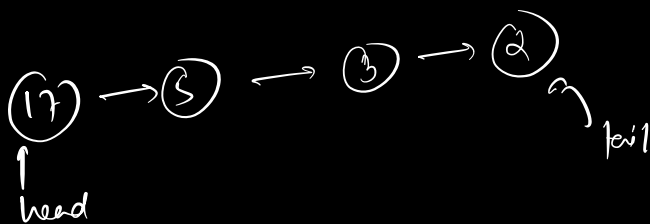
pop()
pop()
push()
pop()



⇒ LL imp of Stack

head = null; tail;

```
push(x) {  
    node* → new Node(x);  
    if (head == null) {  
        head = node;  
        tail = node;  
    }  
    tail->next = node;  
    tail = tail->next;  
    size++;  
}
```



<u>Singly LL</u>		head	tail
		$O(1)$	$O(1)$
	insertion	$O(1)$	$O(1)$
	deletion	$O(1)$	$O(N)$

←—————→

```

push(x) {
    node = new Node(x)
    if (head == null) {
        head = node;
    }
    node.next = head;
    head = node;
}

```

```

pop() {
    if (head != null) {
        x = head.data;
        head = head.next;
        return x;
    }
}

```

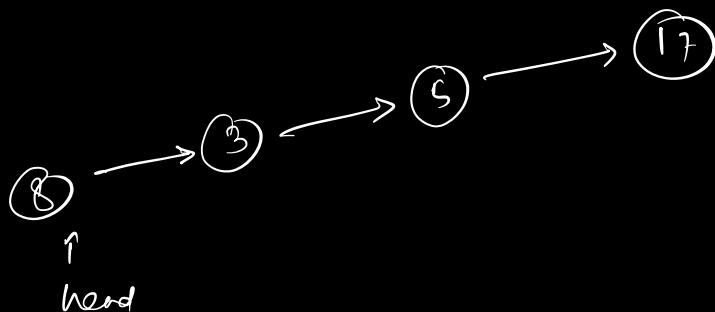
TC $\Rightarrow O(1)$

```

top() {
    if (head != null) {
        return head.data;
    }
}

```

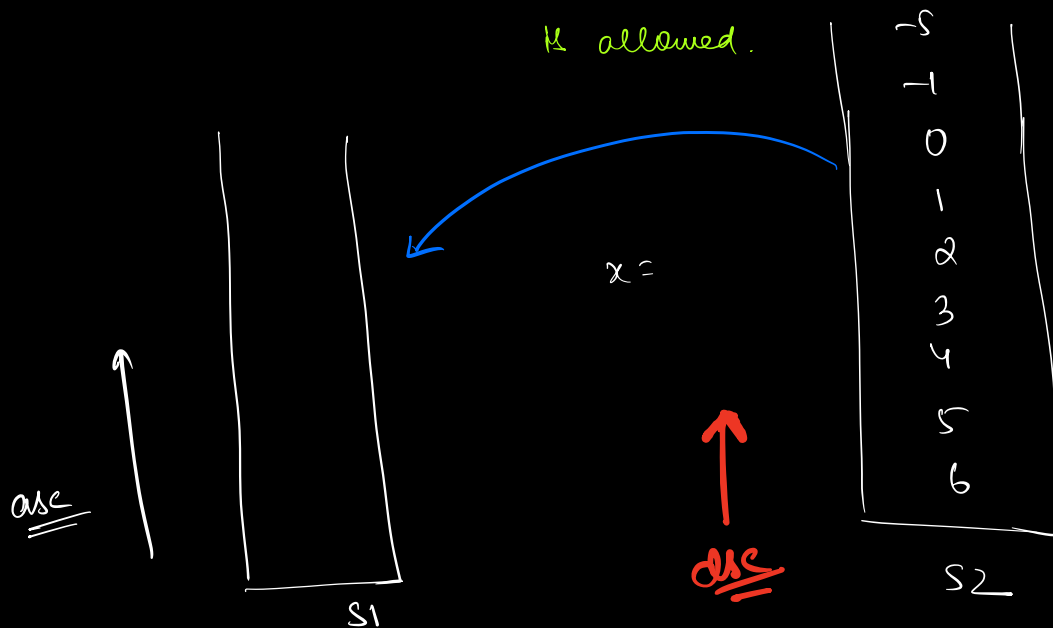
push 17 ✓
 push 5 ✓
 push 3 ✓
 push 2 ✓
 pop → 2
 top → 3
 push 8
 push -1
 pop → -1
 top → 8



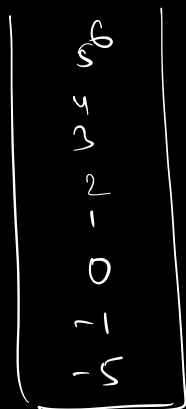
→ you are given a stack →

push	pop	top	isEmpty
			size

Q1. Sort a given stack, only 1 extra stack is allowed, no other data structure or extra space is allowed.



Q2



$$TC \Rightarrow O(N^2)$$

```
while ( !s1.isEmpty() ) {
```

```
    x = s1.pop();
```

```
    while ( !s2.isEmpty() && s2.top() < x ) {
```

```
        temp = s2.pop();
```

```
        s1.push(temp);
```

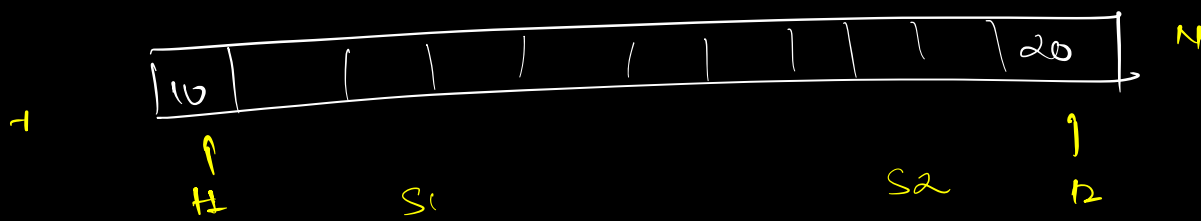
```
    }
```

```
    s2.push(x);
```

```
}
```

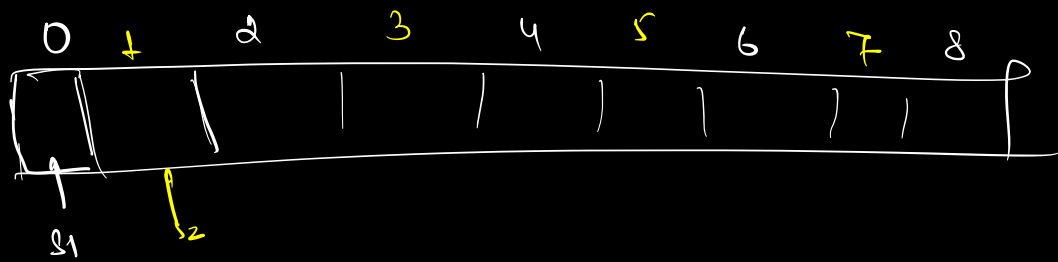
Q2. Implement 2 stacks using a single array.

→ total for
both → N



$S1.push(x)$

$S2.push(20)$



⇒ Implement a stack, which has following operations :-

$O(1)$
 $\underline{\underline{TC}}$

- push()
- pop()
- top()
- getMin() → returns min^m element of stack

push(7)

push(5)

push(2)

getMin() → 2

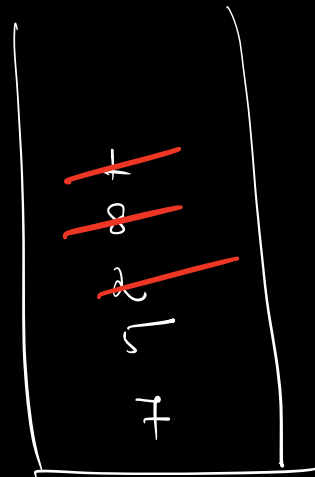
push(8)

getMin() → 2

push(1)

getMin() → 1

pop()



getMin() → 2

pop()

pop()

getMin() → 5

i) extra space (✓)

ii) not using extra space
[Hint]

⇒ extra space ⇒ use another stack to store all min^{ums}.

push(7)

push(5)

push(2)

getMin() → 2

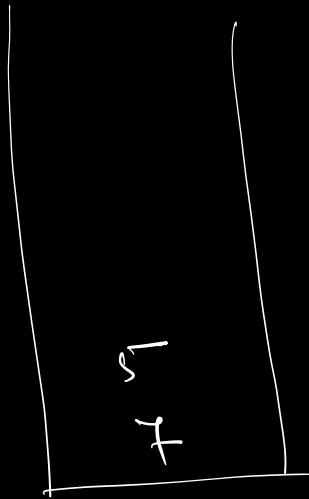
push(8)

getMin() → 2

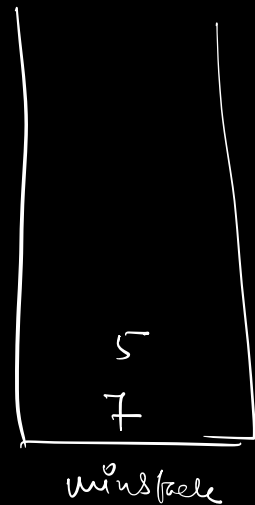
push(1)

getMin() → 1

pop()

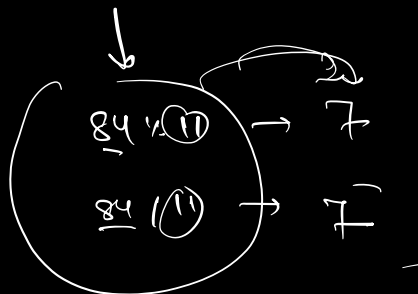


current = 7 < 2



storing when no. ≤ curr. min

$$84 = 7 \times 11 + 7$$



How
date $\rightarrow [1-10]$